Analyse du projet Todo App

Avant de toucher au code, j'ai pris le temps de parcourir la structure du projet pour comprendre ce qui est en place. L'idée ici c'est d'identifier les parties qui pourraient poser problème plus tard : les responsabilités pas clairement séparées, les dépendances trop fortes, et les zones où un bug pourrait passer inaperçu.

---

1. Responsabilités mélangées

Backend

`src/index.js` cumule plusieurs rôles en même temps : il configure les middlewares Express, monte les routes, initialise la base de données *et* gère les signaux système (`SIGINT` , `SIGTERM` , `SIGUSR2`). C'est le point d'entrée mais il fait beaucoup trop de choses.

Les routes (`addItem.js` , `updateItem.js` , etc.) mélangent logique métier et accès aux données. Par exemple `addItem.js` génère un UUID, construit l'objet item *et* appelle directement la couche de persistance — tout ça dans la même fonction. Il n'y a pas de couche service entre les deux.

`persistence/index.js` joue le rôle d'adaptateur (SQLite ou MySQL selon l'env), mais c'est aussi lui qui lit les variables d'environnement pour décider quelle base utiliser. Ce mélange de configuration et de routage rend le module difficile à tester isolément.

Frontend

`TodoListCard.jsx` gère à la fois l'état de la liste, les appels API vers le backend, et le rendu des éléments. Idéalement ce serait trois responsabilités séparées.

`ItemDisplay.jsx` mélange la logique de mise à jour/suppression (appels `PUT` et `DELETE`) avec le rendu visuel de l'item. Un changement dans l'API impacte directement le composant d'affichage.

---

2. Dépendances fortes

Framework

Côté backend, toutes les routes dépendent directement de l'objet `req` / `res` d'Express. Si on voulait remplacer Express par Fastify ou Hono, il faudrait réécrire chaque route — il n'y a aucune abstraction entre le framework et la logique.

Côté client, les composants sont construits autour de React-Bootstrap : la structure HTML et le layout sont mélangés avec le comportement. Changer de lib UI demanderait de toucher à chaque composant.

Base de données

Deux drivers sont utilisés : `sqlite3` et `mysql2` . Chacun a sa propre interface et ses propres particularités. La couche `persistence/` fait un bon travail pour les encapsuler, mais les implémentations ne sont pas interchangeables à 100% (les callbacks SQLite vs les Promises MySQL, la gestion des booléens...).

Le choix de la base se fait au runtime via la variable d'environnement `MYSQL_HOST` . Si elle est absente, on tombe sur SQLite. C'est pratique pour le dev mais ça crée un comportement implicite difficile à tracer.

Filesystem

SQLite écrit le fichier de base par défaut dans `/etc/todos/todo.db`. Ce chemin suppose que le répertoire existe avec les droits d'écriture — ce qui n'est pas garanti dans tous les environnements. En conteneur Docker ça passe, mais en dehors ça peut silencieusement échouer.

En production (image finale), le backend sert les fichiers statiques du frontend depuis `src/static/`. Le build client doit donc avoir été fait *avant* le build backend, ce qui crée un couplage dans le processus de build (géré par le Dockerfile multi-stage, mais pas évident à première vue).

---

3. Zones à risque

Persistance SQLite

C'est probablement la zone la plus fragile. Quelques points concrets :

- **Conversion des booléens** : SQLite ne connaît pas le type `boolean`, il stocke `0` ou `1`. La conversion vers `true` / `false` est faite manuellement dans `sqlite.js`. Si quelqu'un modifie la requête sans penser à ça, les items remontent avec `completed: 0` au lieu de `false` — et ça peut casser le frontend sans erreur explicite.
- **Pas de migration** : le schéma est créé au démarrage avec `CREATE TABLE IF NOT EXISTS`. Ça marche pour une première installation, mais si la structure de la table change (ajout d'une colonne par exemple), les bases existantes ne seront pas mises à jour. Il faudra supprimer la base à la main.
- **Chemin par défaut risqué** : `/etc/todos/todo.db` suppose des droits root. Configurable via `SQLITE_DB_LOCATION` mais cette variable n'est documentée nulle part dans le README.

Couplage API + DB

- **Pas de transaction dans `updateItem.js`** : la route fait deux appels consécutifs — `db.updateItem()` puis `db.getItem()` pour retourner l'item mis à jour. Si la base est dans un état incohérent entre les deux appels, la réponse renvoyée au client peut ne pas refléter la réalité.
- **Pas de validation des entrées** : `addItem.js` accepte n'importe quel `name` sans vérifier sa longueur ou son contenu. La colonne en base est définie comme `varchar(255)`, mais SQLite ne fait pas respecter cette contrainte — une chaîne trop longue passera sans erreur.
- **Les routes connaissent la DB** : il n'y a pas de couche service entre les routes et la persistance. Si on veut ajouter de la logique métier (validation, événements, logs...), il faut l'ajouter directement dans les routes. À terme, ça devient ingérable.

Autres points à garder en tête

- Les credentials MySQL sont en clair dans `compose.yaml` (`secret`, `root`). Acceptable en dev, mais à ne jamais faire pointer vers une vraie base de prod.
- Le pool MySQL est limité à 5 connexions simultanées — pas un problème pour ce projet, mais à surveiller si la charge augmente.
- Aucun endpoint n'est protégé par une authentification. N'importe qui qui a accès au port peut lire, modifier et supprimer tous les items.